

Adaptive IoT Platform for Proactive Wellness

Research Progress Report

Vivek Matta

Research with Professor David Zaretsky

Northwestern University

May 31, 2026

Abstract

This report summarizes progress on an adaptive IoT wellness platform developed for research with Professor David Zaretsky. The project connects a low-cost wearable health band to a native iOS companion application, syncs physiological and activity data over Bluetooth, stores research snapshots locally, and uses an AI coach layer to convert raw metrics into personalized daily actions. The current prototype focuses on the ES02/Veepoo-compatible smart bracelet and a native iOS app called *WatchProbe*. The app has demonstrated watch discovery, verified Bluetooth connection, local health-data sync, AI-backed explanation of synced metrics, and a coach-first interface organized around actions rather than raw sensor dashboards.

1 Research Objective

The research objective is to build an intelligent IoT and mobile ecosystem for proactive wellness. Instead of presenting passive health readings, the system should continuously collect wearable signals, interpret them in context, and produce actionable daily recommendations. The project aligns with the stated research focus of adaptive sensing, on-device or edge intelligence, and personalized wellness planning using AI, IoT, LLMs, and mobile app development.

The long-term vision is a companion platform that can integrate wearable signals such as heart-rate variability, sleep, temperature, blood oxygen, blood pressure, glucose estimates, and activity history. These measurements are treated as contextual wellness signals, not as clinical diagnoses. The system should help users understand patterns, identify uncertainty, and take practical actions such as hydration, movement, sleep recovery, breathing, or training adjustments.

2 Hardware Platform

The current hardware target is a screenless ES02 smart bracelet sourced as a Veepoo-compatible wearable. The listed product capabilities include Bluetooth connectivity, SDK support, IP68 water resistance, magnetic charging, approximately four days of active battery life, and health-signal functions such as ECG, HRV, heart rate, sleep monitoring, body temperature, blood oxygen, blood

pressure, and glucose estimates [1]. The supplier listing identifies the app ecosystem as “G Band” and the Bluetooth version as BT5.3, with Android and iOS compatibility.

The verified development watch used for testing is:

Device name	ES02
Device address	1B:89:F9:42:CF:54
SDK family	Veepoo iOS SDK
Primary test device	Physical iPhone, not simulator



Figure 1: ES02 smart bracelet hardware used for the research prototype.

3 Software System

The active software artifact is the native iOS app, *WatchProbe*. The older Flask chatbot prototype remains useful background, but the current implementation has moved toward a native iOS watch companion because the manufacturer provides an iPhoneOS framework rather than a JavaScript BLE protocol.

3.1 Bluetooth and Watch Sync

The app uses the manufacturer SDK to initialize Bluetooth, scan for compatible watches, connect to the selected device, verify the watch password, and read battery and health data. The connection path uses the shared Veepoo BLE manager and peripheral manager APIs. The app now owns the scan and reconnect flow rather than relying on the SDK’s internal automatic connection behavior, which improves reliability during repeated pairing and demo sessions.

After successful verification, the app remembers the preferred watch and auto-connects on later launches. Local sync snapshots are written to the app’s private application-support storage as timestamped JSON files.

Each sync can include sleep, heart-rate samples, blood oxygen, blood pressure, glucose, temperature, activity, HRV or diagnostic metadata, ECG counts, battery state, and sync diagnostics. The current sync approach runs the SDK base daily read first, then builds JSON from the SDK database snapshot. Direct watch reads remain available as fallback paths when the SDK database path is partial or unavailable.

3.2 AI Coach Layer

After a saved sync, the app builds a compact health context and requests a structured AI analysis. The coach prompt asks the model to explain the latest synced metrics, score major wellness categories, detect timestamp-backed correlations, provide warnings when data is incomplete, and suggest realistic actions. The prompt explicitly avoids medical diagnosis and asks the model to state uncertainty when data is missing, stale, partial, or not clearly correlated.

The app can use Firebase AI Logic when configured, or a local Gemini proxy during development. AI analyses are cached by a SHA-256 fingerprint of the health context and coach personality. If the same health context appears again, the app reuses the previous AI-backed analysis; if the synced data changes, it sends the new context to the AI service.

3.3 Coach-First Interface

The user interface has shifted from an analytics-first dashboard to a coach-first mobile experience. The active iOS shell has four main tabs:

- **Coach:** greeting, goal progress, top checklist items, AI coach message, sync state, and compact armband status.
- **Plan:** daily action cards grouped into Fuel, Move, Mind, and Recovery.
- **Progress:** plain-English summaries first, with sensor analytics available through detail views.
- **Profile:** watch controls, auto-sync, coach personality, reminders, onboarding replay, local AI proxy settings, export, and debug log.

This design supports the research goal of transforming sensor readings into daily actions. Raw metrics remain accessible, but the first-level experience emphasizes recommendations and data quality.

4 Implementation Workflow: SDK Data to App Display

The main engineering work was connecting the vendor SDK, the physical watch, the local data model, and the SwiftUI interface into one repeatable flow. The process was implemented and tested in stages so that each layer could be verified before depending on it for the next layer.

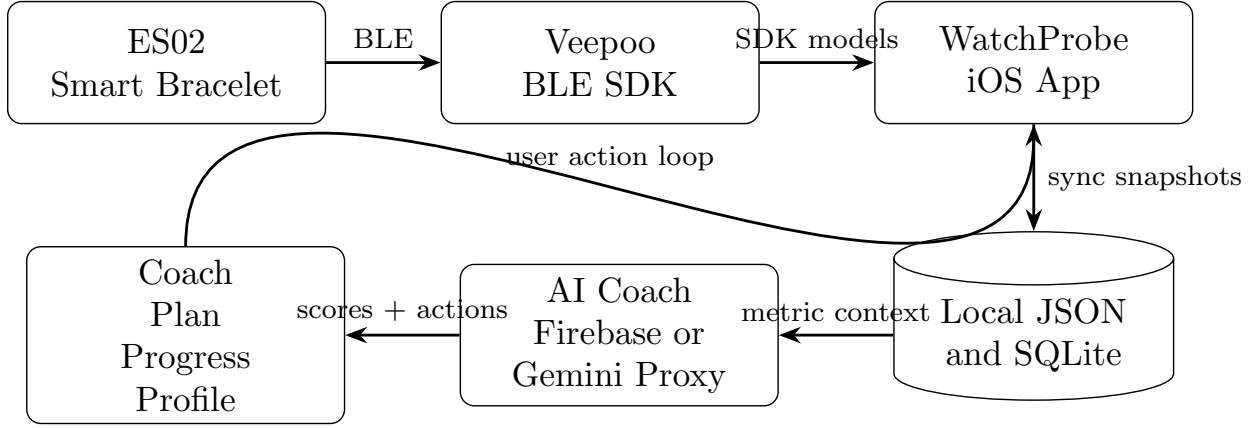


Figure 2: System architecture for the adaptive IoT wellness platform.

4.1 1. SDK Setup and App Integration

The first step was to move from the older web-chat prototype to the native iOS project because the wearable exposes its supported data through the Veepoo iOS SDK. The SDK framework and SDK documentation were added to the app project, then the app was configured to run on a physical iPhone because Bluetooth Low Energy watch pairing and the vendor iPhoneOS framework cannot be validated in the iOS simulator.

The iOS app initializes the SDK’s shared Bluetooth manager at launch and requests the required Bluetooth permission from iOS. After initialization, the app owns the scan, connect, verify, sync, and reconnect lifecycle. This made the connection behavior easier to debug than relying only on the SDK’s automatic connection behavior.

The essential SDK setup is small: the app gets the SDK singleton and attaches the peripheral manager before any Bluetooth operation starts.

```

let bleManager = VPBleCentralManage.sharedBleManager()
bleManager.peripheralManage = VPPeripheralManage.shareVPPeripheralManager()

```

4.2 2. Watch Discovery and Connection Verification

The watch connection workflow starts with an explicit scan for nearby Veepoo-compatible devices. The scan results are filtered by the known hardware target, especially the ES02 device name and the verified watch address 1B:89:F9:42:CF:54. Once the correct watch is found, the app starts the SDK connection flow and waits for the SDK callback indicating that the peripheral is connected.

The scan and connection flow uses the SDK’s scan callback to collect a `VPPeripheralModel`, stops scanning once the target watch is found, and then passes that model into the SDK connection call.

```

VPBleCentralManage.sharedBleManager()
    .veepooSDKStartScanDeviceAndReceiveScanningDevice { model in
        guard model.deviceName == "ES02" else { return }
        selectedWatch = model
        VPBleCentralManage.sharedBleManager().veepooSDKStopScanDevice()
    }

VPBleCentralManage.sharedBleManager()
    .veepooSDKConnectDevice(selectedWatch) { connectState in
        switch connectState {
        case .BleConnectSuccess:
            connectionStatus = "Connected"
        case .BleVerifyPasswordSuccess:
            connectionStatus = "Verified"
        default:
            connectionStatus = "Waiting"
        }
    }
}

```

After the Bluetooth link is established, the app verifies the watch password state through the SDK. Password verification is treated as a separate connection milestone because a device can be visible over Bluetooth without being ready for protected data commands. The app then reads the battery state as a lightweight first command. A successful battery read confirms that the app can send commands to the connected watch and receive parsed SDK responses.

Battery reading was used as a low-risk first command after verification because it confirms that the app can communicate with the connected watch before starting larger health-data reads.

```

VPBleCentralManage.sharedBleManager()
    .peripheralManage
    .veepooSDKReadDeviceBatteryInfo { isPercent, isLowBattery, battery in
        batteryPercent = isPercent ? battery : nil
        batteryWarning = isLowBattery
    }
}

```

Verification check	Purpose
Scan result found	Confirms that the iPhone can see the ES02 over Bluetooth.
SDK connected callback	Confirms that the BLE link is established.
Password verified	Confirms that the watch accepts app commands.
Battery read succeeds	Confirms that SDK command/response parsing is working.
Preferred watch saved	Confirms the app can auto-reconnect on later launches.
JSON snapshot written	Confirms that synced data reached app storage.

4.3 3. Reading Data from the SDK

Once the watch is connected and verified, the app runs the SDK's base daily sync. This call is important because the Veepoo SDK stores many readings in its own local database after a sync. The app then reads from that SDK database snapshot instead of assuming every metric should be pulled directly from the watch in real time.

The data retrieval flow is intentionally serialized. The vendor SDK warns that data commands should not run concurrently, so the app avoids overlapping watch operations. This reduces partial payloads, command failures, and hard-to-debug sync states.

The main data read starts with the SDK daily sync. The callback reports the read state, total number of days, current day, and progress for the current day. The app waits for completion before extracting data from SDK persistence or running any fallback reads.

```
VPBleCentralManager.sharedBleManager()
    .peripheralManager
    .veepooSdkStartReadDeviceAllData { state, totalDay, currentDay, progress in
        syncProgress = "\(currentDay)/\(totalDay) \(progress)%"

        if state == .complete {
            loadSyncedMetricsFromSDKDatabase()
            writeResearchSnapshot()
            refreshCoachAndProgressViews()
        }
    }
```

For data types where the SDK exposes a direct watch-storage read, the app can use those calls as targeted fallback paths. The project used these reads cautiously and one at a time.

```
peripheral.veepooSDK_readStepDataWithDayNumber(0) { stepDict in
    normalizeStepSummary(stepDict)
}

peripheral.veepooSDK_readSleepDataWithDayNumber(1) { sleepSegments in
    normalizeSleepSegments(sleepSegments)
}

peripheral.veepooSDK_readBasicDataWithDayNumber(0, maxPackage: 0) {
    basicArray, totalPackage, currentPackage in
    appendHeartRateAndActivitySamples(basicArray)
}
```

After the base daily sync completes, the app normalizes SDK data into research-friendly metric groups:

- Sleep records, sleep duration, interruptions, and a local sleep score.
- Heart-rate samples and summary values.
- Blood oxygen, blood pressure, glucose estimate, temperature, and HRV records when available.

- Activity metrics such as steps, calories, distance, and active time.
- ECG availability or counts when the SDK exposes them.
- Battery state, device identity, timestamps, and sync diagnostics.

When an SDK database path is incomplete, direct watch-read fallback paths remain available. These fallback reads are useful for testing individual data types, but the daily SDK database sync is the preferred path for repeatable research snapshots.

4.4 4. Saving Research Snapshots

After SDK data is normalized, the app writes a timestamped JSON snapshot under the app sandbox. Each file includes the device identifier, sync time, available metrics, and diagnostics describing which data types were present, partial, or missing. This snapshot format was added so the research data can be inspected independently from the app UI.

The snapshot layer also gives the AI coach a stable input. Instead of asking the model to reason over raw SDK objects, the app converts the synced watch records into a compact health context with plain keys, timestamps, values, and data-quality notes.

The app-level code treats SDK models as an input layer only. After parsing, the data is converted into a stable JSON shape that the UI and AI coach can both consume.

```
let snapshot = WatchResearchSnapshot(
  deviceName: "ES02",
  syncedAt: Date(),
  batteryPercent: batteryPercent,
  sleep: normalizedSleep,
  heartRate: normalizedHeartRate,
  oxygen: normalizedOxygen,
  activity: normalizedActivity,
  diagnostics: syncDiagnostics
)

try snapshotWriter.write(snapshot)
```

4.5 5. Connecting the Data to the AI Coach

The app builds an AI context from the latest saved snapshot. That context includes current metrics, recent trends when available, missing-data warnings, and the user's selected coach personality. The app calculates a SHA-256 fingerprint of this context so repeated syncs with unchanged data can reuse a cached AI response.

When the context changes, the app sends the new summary to Firebase AI Logic or the local Gemini proxy. The returned structured analysis is converted into app-level objects: wellness scores, metric explanations, warnings, correlations, and recommended actions. The model is instructed to stay within wellness coaching language and to avoid medical diagnosis.

4.6 6. Displaying the Data in the App

The final step was connecting the saved sync data and AI analysis to the SwiftUI screens. On app launch, the app loads the most recent JSON snapshot and uses it to populate the shared app state. After a new sync completes, the same loading path refreshes the interface so the user sees the newest watch data without restarting the app.

The display layer is organized by user task:

- The **Coach** tab shows the connection state, last sync state, top recommended actions, and the AI coach explanation.
- The **Plan** tab turns the AI and local rules into checklist-style actions such as hydration, walking, breathing, recovery, and sleep preparation.
- The **Progress** tab exposes health metrics and plain-English summaries so raw readings remain reviewable.
- The **Profile** tab contains watch controls, sync settings, export tools, coach configuration, and debug information.

This structure separates the raw data pipeline from the user experience. The SDK and JSON snapshot layer handle data collection and traceability, while the SwiftUI tabs translate that data into status, explanations, trends, and daily actions.

At the UI level, the app does not display raw SDK objects directly. It displays normalized app state built from the latest snapshot and the latest coach analysis.

```
struct CoachView: View {
    @ObservedObject var model: WatchCoachModel

    var body: some View {
        VStack(alignment: .leading) {
            Text(model.connectionStatus)
            Text(model.lastSyncSummary)
            Text(model.coachMessage)
            ForEach(model.topActions) { action in
                ActionCard(action: action)
            }
        }
    }
}
```

4.7 7. End-to-End Test Procedure

The working prototype was verified using the following repeatable procedure:

1. Build and run *WatchProbe* on a physical iPhone.
2. Grant Bluetooth permission and start the app's watch scan.

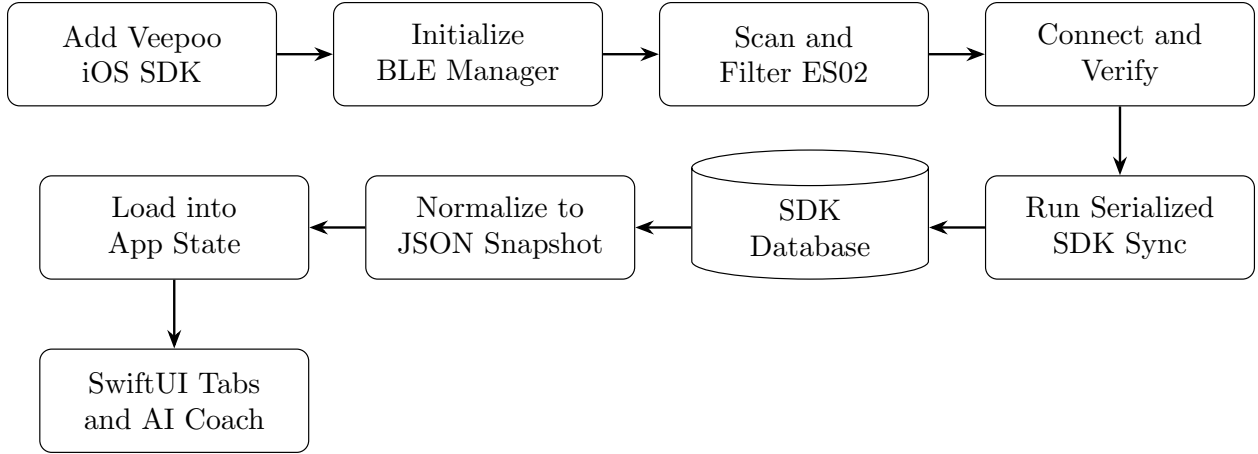


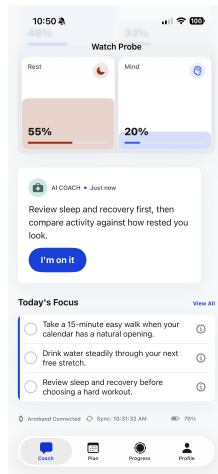
Figure 3: Implementation path from vendor SDK setup to data display in the app.

3. Confirm that ES02 / 1B:89:F9:42:CF:54 appears in scan results.
4. Connect to the watch and wait for the SDK connected callback.
5. Verify the watch password state.
6. Read the battery value as the first command-level confirmation.
7. Run the base daily SDK sync.
8. Confirm that a timestamped JSON file is written to local app storage.
9. Reload the latest snapshot into the app state.
10. Check that the Coach, Plan, Progress, and Profile tabs reflect the latest connection state, sync timestamp, metrics, AI explanation, and actions.

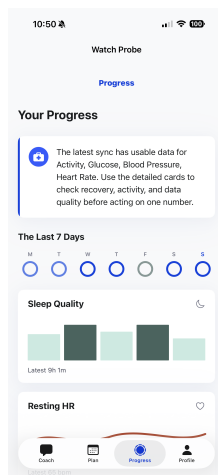
5 Implementation Progress

The prototype currently demonstrates the following milestones:

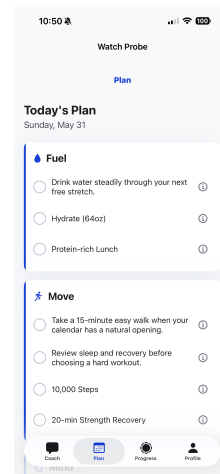
- Physical watch connection is verified for ES02 / 1B:89:F9:42:CF:54.
- The app can scan for the watch, connect, verify password state, read battery status, and persist the preferred watch.
- Watch data is saved as local JSON snapshots for research inspection and later analysis.
- The app performs SDK database sync first and uses direct-read fallback paths for partial sync cases.
- The UI loads the latest saved JSON at launch and refreshes after successful syncs.
- The AI coach produces structured summaries, metric explanations, warnings, scores, correlations, and suggested actions.



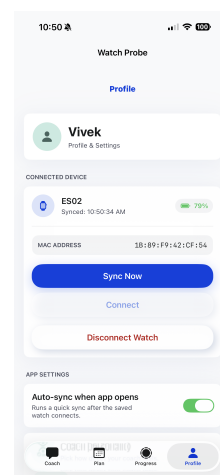
Coach



Progress



Plan



Profile

Figure 4: WatchProbe app screenshots from the current coach-first interface, showing the Coach, Plan, Progress, and Profile tabs.

- The Plan checklist supports actionable tasks such as hydration, walking, breathing, sleep preparation, and recovery-aware workout suggestions.
- Accepted reminder-style actions can schedule local push notifications.
- A local sleep score estimates duration, consistency, and interruptions using watch sleep data.

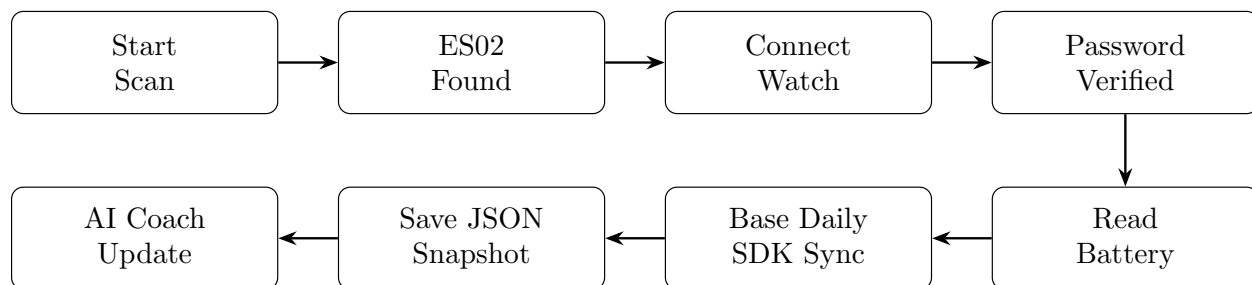


Figure 5: Watch connection and sync workflow used by the prototype.

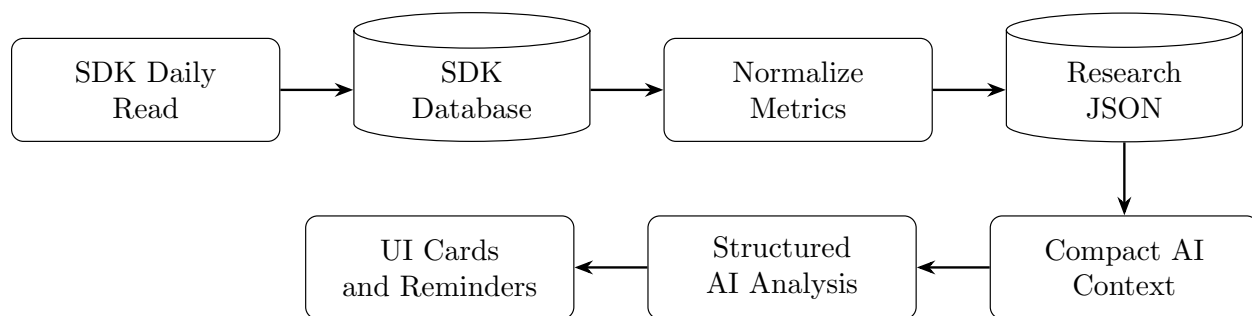


Figure 6: Data pipeline from watch sync to AI-backed daily actions.

6 Current Results

The main result so far is a working end-to-end prototype that connects a commercial wearable device to an AI wellness coaching interface. The project has moved beyond static mock data: the app can use real watch sync snapshots, preserve multi-day records, expose metric history, and generate coach actions from the available health context. This validates the core research direction that low-cost wearable sensors can feed a personalized action layer through a mobile app and LLM-based reasoning.

The system also shows why careful data-quality handling is important. Wearable measurements can be missing, partial, unsupported, delayed, or vendor-SDK dependent. The current AI prompt and UI account for this by surfacing uncertainty rather than treating every value as authoritative.

7 Challenges and Limitations

- **SDK reliability:** The vendor SDK warns that data commands are not concurrent, so all watch operations must be serialized. Some historical SDK paths were fragile on ES02 payloads.
- **Measurement quality:** The watch provides consumer wellness estimates. Blood pressure, glucose, ECG, and related measurements should be treated as non-diagnostic context.
- **Automatic measurement settings:** The app-side auto-sync timer runs every 10 minutes while open and connected, but the app does not yet fully audit or configure every watch-side automatic measurement interval.
- **Data history:** More consecutive days of real synced data are needed before the AI coach can reliably compare a user against personal baselines.
- **Production backend:** The local Gemini proxy is useful for development, but a production research deployment would need a secure backend and data-governance plan.

8 Next Steps

1. Add an automatic measurement settings audit for heart rate, blood pressure, HRV, SpO2, glucose, and temperature.
2. Add optional controls to enable or configure supported watch-side automatic measurement intervals.
3. Continue collecting multi-day sync snapshots to evaluate sleep, activity, and recovery trends.
4. Add per-data-type sync watermarks after the returned payload shapes and timestamps are stable.
5. Improve export and reporting workflows for research review.
6. Add additional validation notes comparing UI values against the raw JSON snapshots during demo runs.

9 Conclusion

The current research prototype establishes a practical foundation for an adaptive IoT wellness platform. It connects a screenless health bracelet to a native iOS app, captures real wearable data, stores sync snapshots, and converts the latest health context into action-oriented AI coaching. The added SDK-to-display workflow shows how the data moves from the Veepoo SDK through verified Bluetooth connection, serialized sync, local research snapshots, AI analysis, and finally into the Coach, Plan, Progress, and Profile tabs. The next research stage should focus on improving automatic measurement configuration, collecting more longitudinal data, and evaluating whether the coach-first interface helps users understand and act on their wellness signals more effectively than a raw-metrics dashboard.

References

- [1] Alibaba.com. “2026 ES02 Screenless Smart Bracelet Body Temperature ECG HRV Heart Rate Sleep Monitoring Fitness Tracker Sports Modes Smart Band.” Accessed May 31, 2026. https://www.alibaba.com/product-detail/2026-ES02-Screenless-Smart-Bracelet-Body_1601721653285.html
- [2] Veepoo iOS SDK documentation and project integration notes included with the SDK materials.
- [3] WatchProbe iOS app source code and SDK integration in the project repository.